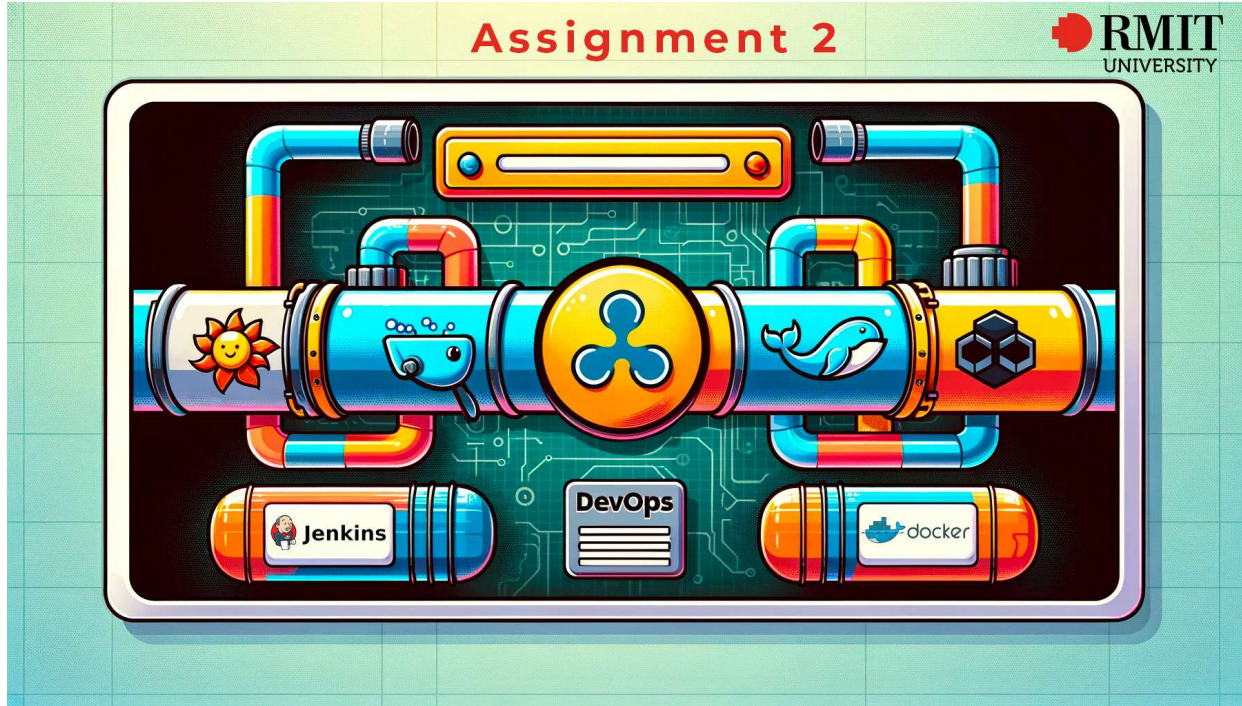


Assignment 2



Course code and name: COSC2767|COSC2805 - Systems Deployment and Operations

Length: ~4 weeks

Type: Group

Deadline: Wednesday, Week 12.

Feedback mode: Not available because this is the second last assessment

Late work: A penalty of 10% per day is applied for up to 5 days late. After 5 days, the penalty is 100%.

Learning Objectives Assessed

- Understand the importance of the DevOps role within modern Agile Software Engineering process.
- Understand the issues associated with deploying software systems, including code repository workflows and how to manage risk in a continuous deployment environment.
- Be capable to using modern tools to manage flexible deployment across a range of environments
- Understand the issues involved in ensuring system qualities to meet the non-functional requirements of a system.
- Be capable of working closely with developers in a Software Development team and managing the deployment and operation of software systems produced by that team

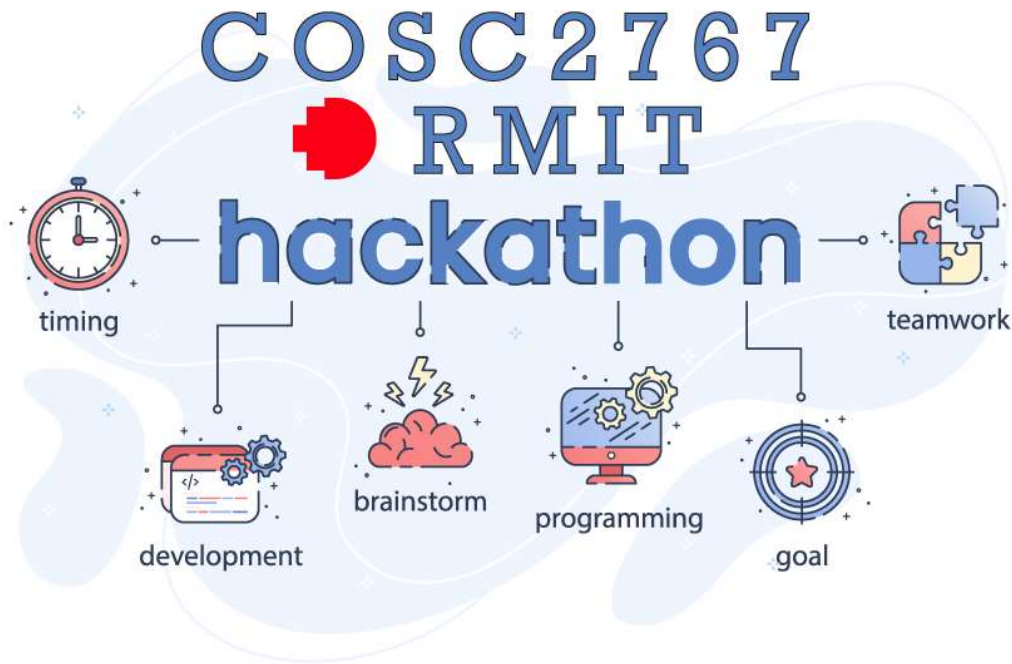
Ready for Life and Work

- Analyze and design a DevOps solution for a given code integration and deployment problem.
- Implement the pipeline using Jenkins.
- Test and debug the pipeline.
- Manage source code using GitHub.
- Learn to manage different servers on AWS to simulate the development and production pipeline.

Assessment Details

 **AI CONDITION: This assessment uses Condition 3 - Bounded Process AI.**

AI may be used only for the approved process purposes listed in the "AI Condition for this Assessment" section below. **Your team must submit one completed Condition 3 Declaration with your work, even if no member used AI.** A submission without the declaration is incomplete: your submission date becomes the date on which both the work and the declaration have been received, and the late penalty below is applied from that date.



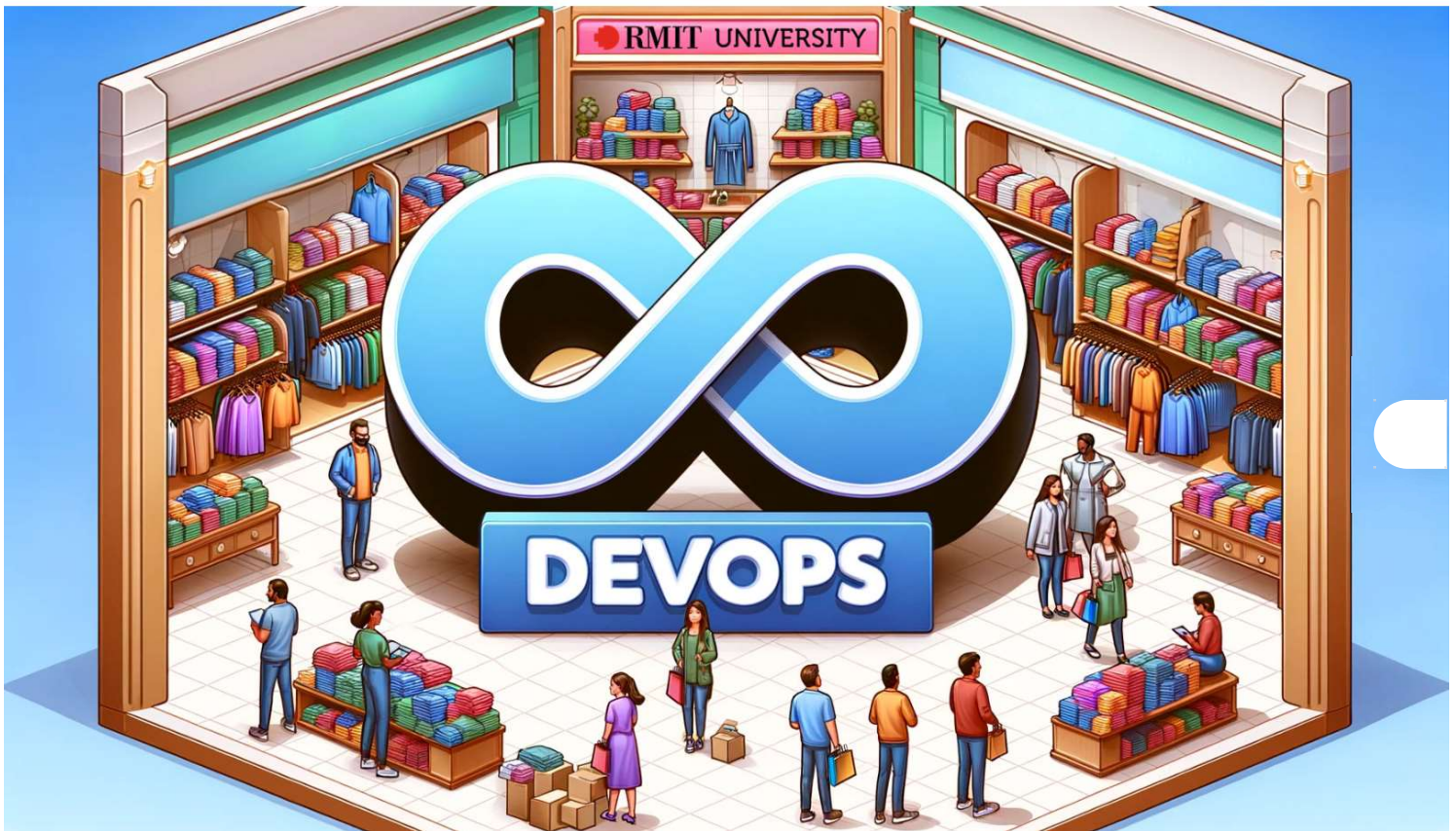
In Australian universities, one of the best ways for students to get real coding experience is to join a team-project-based hackathon hosted by tech companies to solve problems they deeply care about.

The name comes from “hack” and “marathon”, which should give you a good idea of the meaning of this word. Thus, a hackathon is a coding event that aims to create, innovate, improve and develop a software solution as an attempt to resolve an existing problem in society.

Thus, the nature of this assignment 2 is to provide you a taste and experience of a 4-week tech hackathon.

In this assignment your team takes on the role of a DevOps consultancy. You are given a working three-tier web application and a developer who has no pipeline, and you must make deploying, testing, scaling, observing and rolling back that application boring, repeatable and automatic.

Problem Statement



Scenario

Tom is a web developer responsible for building and maintaining the **RMIT E-commerce Store**, which sells RMIT merchandise. Tom is a capable web developer, but he has no experience with DevOps or CI/CD. He struggles to deploy updates efficiently, to track what has been deployed, and to be confident that a release is reliable and scalable enough for real production traffic.

Without a pipeline, Tom faces the following problems.

- **Manual deployment.** Every release is done by hand and is slow and error-prone. On this stack a manual deploy is a sequence of steps that must happen in the right order: pull, install dependencies, run database migrations, collect static files, rebuild the frontend bundle, restart the application server. Skipping any one of them breaks the site in a different way.
- **No test automation.** Testing is inconsistent or skipped under time pressure. **The repository ships with no test suite at all**, so nothing currently stands between a bad commit and the live store.
- **No deployment visibility.** Tom cannot easily tell what changed, when it went out, or whether it succeeded, which makes troubleshooting and accountability difficult.
- **Scalability problems.** The application writes uploaded product images to local disk, so simply running a second server breaks the storefront in a way that looks intermittent and random.

- **No rollback path.** When a release fails there is no reliable way back to the last good state. Database migrations make this harder than it looks: rolling the code back does not roll the schema back.
- **High release risk.** Every deployment is risky enough that Tom delays improvements rather than shipping them.
- **Poor collaboration.** Changes from multiple contributors are not integrated continuously, causing conflicts and delays.
- **Stress and burnout.** The manual, fragile workflow consumes the attention Tom would rather spend on features.

Tom has engaged your **DevOps team** to build a CI/CD pipeline that lets him develop and release updates to the RMIT Store efficiently and reliably.



The Application

The site is built on a **Django + Vue** stack, chosen for Django's maturity in building secure REST APIs and Vue's strength as a modern single-page frontend. It has three independently deployable tiers.

Tier	Technology	Runs as
Frontend	Vue 3 single-page application, built with Vite	A build artifact , a folder of static files served by a web server such as nginx
Backend	Django 5 + Django REST Framework	A WSGI application served by gunicorn
Database	PostgreSQL 14+	A stateful service with a schema managed by migrations

The repository contains the current codebase and setup instructions:

<https://github.com/TomHuynhSG/COSC2767-RMIT-Store-Django-Vue> 

Your first task is to deploy the site **locally, then on a single EC2 instance**, so you understand how it works before you automate anything. The repository README walks through both, and Plan A and Plan B are written step by step.

Why this stack makes the deployment problem interesting

This is not a toy application. Several of its properties exist specifically to create the operational problems this course is about. Read these before you design your pipeline, because each one is a decision your pipeline has to make.

1. **There is a build step.** The frontend is not shipped as source; Vite compiles it into a `dist/` folder. Where does that build run: on the server, or in CI? Building on a `t3.micro` can run the instance out of memory, which is itself a lesson about what belongs in a pipeline.
2. **There are database migrations.** The schema is versioned, and `python manage.py migrate` must run as part of every deploy, in the right order relative to the code that depends on it. Your rollback strategy has to account for this.
3. **The frontend is configured at runtime, not at build time.** The API URL is read from `client/public/config.js` when the page loads, so **one built artifact can be promoted from testing to production unchanged**. Rebuilding per environment is the anti-pattern this design lets you avoid.
4. **Uploaded product images are state on the instance.** By default they are written to `server/media/` on local disk. The moment you run a second replica, images uploaded to one container 404 from the other. Fixing this properly, with shared object storage, is part of the work.
5. **Static files must be collected.** `python manage.py collectstatic` has to run before the Django admin and the browsable API render correctly.

6. **Configuration comes entirely from the environment.** Settings are split into dev and prod modules selected by `DJANGO_SETTINGS_MODULE`, and production **refuses to start** without a real `SECRET_KEY` and `ALLOWED_HOSTS`. Getting secrets to a container without committing them is your problem to solve.
 7. **ALLOWED_HOSTS and load balancers interact badly.** A health check arrives with a `Host` header of the instance's private IP; if that is not allowed, Django returns 400 and your target group marks a healthy instance unhealthy.
 8. **Health and version endpoints already exist.** `GET /healthz/` (liveness), `GET /readyz/` (readiness, which checks the database and storage), and `GET /api/version/` (which build is running). Use them.
-



Objective

Design and build a CI/CD pipeline that automates the development and deployment process for the RMIT Store, and that demonstrably delivers **automation, efficiency, reliability, scalability and repeatability**.



Core Requirements

All nine requirements below must be addressed.

Where a requirement names a tool, the following table states whether that tool is mandatory or an example. Substituting a mandated tool is not permitted; substituting an example tool is, provided you justify the choice in your report.

1. Cloud integration

Leverage and maximise the use of **AWS services** for performance and scalability and others as appropriate.

2 Containerised Services

Deploy the frontend and backend tiers as **Docker containers** to improve reliability, scalability and consistency across environments.

The database tier may either run as a container or be replaced by a managed service such as **AWS RDS**.

State and justify your choice in the report.

Consider carefully where **uploaded media lives** once more than one backend container is running.

3 Continuous Integration

Automate CI so that **every GitHub update triggers the pipeline**, building both tiers, running the test suites, and reporting the result.

4 Continuous Deployment

Deploy updates to a **staging server first**. On successful testing, updates should be deployed to the **production server automatically**. If an update fails during testing, the team must be notified and **production must not be updated**.

5 Testing framework

Important. This repository deliberately ships with **no tests at all**: no unit tests, no integration tests, no end-to-end tests. Writing them is part of the assessment. You choose the tooling, the structure, the coverage target and how the suites run in your pipeline, and you justify those choices.

Implement a range of testing methodologies (**Unit, Integration and Web UI**) appropriate to this problem statement and this website, and **justify each choice**.

Ideas that map onto this application specifically:

- **Unit:** the payment gateway logic in isolation (card validation, brand detection, expiry, decline codes); server-side tax and total calculation; frontend cart state (merging lines, clamping quantity against stock); the card-input formatting component.
- **Integration:** that the shop endpoint actually returns products, proving the API-to-database connection; that placing an order decrements stock and computes the total server-side; that a declined card leaves no order behind and takes no stock; that authorisation holds, so an anonymous visitor, a member, a seller and an administrator each get the status codes they should from every endpoint.
- **Web UI / end-to-end:** that the shop page renders products, proving frontend → backend → database is connected end to end; that a user can register, sign in and sign out; that a product page shows its image, price, stock and reviews; that the full purchase journey completes; that a seller can add a product and see it appear on the storefront.
- **Smoke / deployment tests:** GET /healthz/ and GET /readyz/ return healthy, and GET /api/version/ reports the build you just deployed. These are the cheapest and most valuable tests in the whole pipeline.

Also test the basic failure modes: **broken connections between frontend, backend and database**, and misconfiguration that only shows up in a deployed environment (wrong API URL, wrong ALLOWED_HOSTS, missing static files, ...).

The **Verifying a deployment** section of the repository README lists acceptance journeys written to be converted directly into automated end-to-end tests.

6 Automation tool

Use a tool such as **Jenkins** to automate the stages of the development and deployment process. Your pipeline should build the frontend bundle, install backend dependencies, run migrations, run the test suites, publish the results, build and push container images, and deploy.

7 Configuration management

Use a tool such as **Ansible** to manage server configuration: installing runtimes, configuring web server, and so on.

8 Automated alerting

Implement **email notifications** to the team for build, test and deployment failures.

9 Orchestration

Use **Docker Swarm or Kubernetes** to deploy, scale and manage the containerised application.

Advanced Requirements

To build the strongest pipeline for this problem statement, also complete the following.

1 Infrastructure as Code

Use **AWS CloudFormation** to define and provision your cloud infrastructure through code, including EC2 instances, the S3 bucket for product images, IAM roles and policies, security groups and load balancers,...

2 Monitoring and alerting

Integrate monitoring and alerting (for example **Prometheus**, **Grafana** or **CloudWatch**) for real-time insight into application health and performance. The existing `/healthz/` and `/readyz/` endpoints are ready for probes; exposing an application metrics endpoint is your job.

3 Advanced deployment strategy

Use a **canary** or **blue-green** strategy to reduce downtime and deployment risk. `GET /api/version/` reports the running build, so you can verify which version is serving traffic from outside the instance.

4 Additional enhancements

You are encouraged to identify and implement further features or tools that improve the pipeline's automation, efficiency, reliability, scalability and repeatability, and that address Tom's pain points. Ideas well suited to this stack:

- **Secrets management:** AWS Secrets Manager, SSM Parameter Store, Jenkins credentials, Ansible Vault or Kubernetes Secrets instead of `.env` files on disk.
 - **Zero-downtime migrations:** a documented, tested procedure for schema changes that are safe to apply while the previous version is still serving.
 - **Automated rollback:** triggered by a failing health check or an error-rate threshold after deploy.
 - **Database backup and restore**, with a restore you have actually rehearsed.
 - **Static analysis and security scanning** in CI: linting, dependency vulnerability scanning, container image scanning.
 - **A CDN** in front of the static bundle and the S3 media bucket.
 - **Log aggregation:** the application emits structured logs to the console with a configurable level, which is exactly what a log shipper expects.
 - And so on...
-

Getting Started

- 1. **Read the README** in the repository. It documents the architecture, the environment variables, and six deployment plans ranging from "run it on your laptop" (Plan A) to "build a real pipeline" (Plan F).
- 2. **Deploy it manually first**, locally and then on a single EC2 instance, before automating anything. You cannot write an Ansible playbook or a Dockerfile for a process you have not performed by hand, and every manual step you find painful is a step your pipeline should remove.
- 3. **Seed the demo data** so you have something to test against the catalogue of 15 categories, 16 brands, 47 photographed products and around 160 reviews, plus optional demo accounts.
- 4. **Explore the API** at /api/docs/. It is an interactive Swagger UI, and it works before the frontend exists.
- 5. **Then start building the pipeline.**

Useful endpoints

Endpoint	Purpose
GET /healthz/	Liveness: is the process alive? Touches nothing.
GET /readyz/	Readiness: checks the database and storage; returns 503 if either is down.
GET /api/version/	Reports APP_VERSION and GIT_COMMIT of the running build.
/api/docs/	Interactive OpenAPI documentation.
/admin/	Django admin, useful for inspecting a deployment directly.

Testing payments

The payment gateway is **simulated**. There is no provider, no account, no API key and no outbound network call, so it cannot fail in CI or on a private subnet. It accepts the familiar test card numbers:

Card number	Result
4242 4242 4242 4242	Approved
4000 0000 0000 0002	Declined

A declined card leaves no order behind and takes no stock, which makes the decline numbers safe to use repeatedly in automated tests and deployment checks. The full table is in the repository README.

Scope

The pipeline is the deliverable, not the application. You are not expected to add storefront features. You **are** expected to make deploying, testing, scaling, observing and rolling back this application boring, repeatable and automatic, which is precisely what Tom is missing.



The Document Requirements

Keep in mind, Tom is a normal fullstack web developer of the website and request for the pipeline from your team. Therefore, there is no right or wrong answer for your CI/CD pipeline solution, and that's what your team need to figure out together based on the requirements.

You need to provide the project document to Tom to explain about your CI/CD pipeline regarding use cases and purposes which fit his development and deployment requirements.

The document should be in **PDF** file format.

The document needs to includes:

- **Title Page:**
 - Title of the Assignment
 - Course Title
 - Lecturer's Name
 - Your Team Member Names and Student IDs
 - Submission Date
- **Table of Contents:**
 - List of all sections and subsections along with page numbers.
- **The CI/CD Pipeline Solution Description:**
 - The brief problem statement.
 - The proposed solution for the pipeline.
 - Introduction to the tools and technologies used in the assignment (Jenkins, Dockers, so on).
 - Features of this pipeline.
 - Anything else you want to include.
- **Main Requirement Section:**
 - For each completed main requirement, please provide description, configurations and screenshots as proofs.
 - Explanation of the chosen settings and configurations if needed.
 - Your design decision and why.
 - Screenshots are important to show the pipeline can pass required use cases.
- **Testing strategy** and the justification for each kind of test.
- **Advanced Requirement Section (if applicable):**
 - For each completed advance requirement, please provide description, configurations and screenshots as proofs.
 - Explanation of the chosen settings and configurations if needed.
 - Your design decision and why.
 - Screenshots are important to show the pipeline can pass required use cases.
- **Application Flow (Diagram):**
 - Includes diagram(s) how different components interact with each other in the Pipeline from end to end. Also, it should also reflect how to separate the website into different services (frontend,

- backend, and database).
- Consider these two diagrams are big pictures overview of your team's work.
 - Runtime application architecture.
 - CI/CD pipeline sequence.
- **Known Bugs/Problems:**
 - Provides the list of known bugs/problems you haven't fixed.
- **Self Evaluation and Reflection of your CI/CD pipeline:**
 - Provide self-evaluate your own CI/CD pipeline solution with your completed requirements.
 - Reflect upon the industry practices from the guest lecture talk in the semester.
 - The brief reasons why your team think this is the most appropriate solution to solve this integration and deployment problem.
 - Evaluate the effectiveness of the pipeline in handling failure recovery, rollback scenarios, and scalability. Reflect on any challenges faced in implementing these features and how they were addressed.
 - The aspects of automation, efficiency, reliability, scalability and repeatability of your pipeline setup are highly important. Therefore, it is vital for you to convince the lecturer that you have achieved these aspects in your pipeline with main and advance requirements.
- **Project Responsibilities:**
 - **Provides the table of the actual responsibilities and workload (%) of each team member regarding to project subtasks (based on the project task breakdown). (This should be discussed as a team together and finalized by the team leader)**
- **Conclusion:**
 - Summary of the key learnings from the assignment.
 - Reflection on the experience and the skills acquired in relation to DevOps practices.
- **References:**
 - List of any references or resources used for completing the assignment.
- **Appendices (if necessary):**
 - Any additional information, scripts, or screenshots that support the document but are not part of the main content.

This structure should provide a comprehensive report fulfilling the document requirements of the assignment while reflecting a thorough understanding and application of the DevOps practices taught in the course.

Please Note: The report should be formatted in a professional and well organized manner as such having a cover page, table of content, page numbers, easy-to-read formats and fonts. The report page limit is 25 pages excluding cover page, table of content, Project Responsibilities, reference and appendix.

If the report contains a lot of screenshots or diagrams or visual aids, then you can put them in the appendix section so they do not count toward the page count. To be clear about what is marked: place **one decisive screenshot** per requirement in the body next to the explanation it evidences, and put supporting captures in the appendix with a cross-reference from the body. A requirement whose only evidence is an unreferenced appendix image will be marked as undemonstrated.

The Video Presentation Requirements

You need to make **one single video** explaining how you analyze, design, and implement the system and a working demo which shows the features completed in your CI/CD Pipeline with completed main and advance requirements. Also, it should show a commit flowing through the pipeline to staging and then production, a failing build stopping a release, and a rollback or advanced deployment strategy in action.

- The whole length of the demo video is within 20 mins.
- Your team can structure the video presentation to have two parts as following:
 - The project detail presentation: team members take turn to present about the project, which reflect the required above structure of the final document. Your team can either present with powerpoint slides or with the final document.
 - The CI/CD pipeline demo: Showing the actual demo of the application to demonstrate main & advance requirements of the pipeline.
- The demo must include use cases & scenarios:
 - **Very important to have a quick walkthrough or live demo for all of the live components of your main and advance requirements as proofs.**
 - Display the website running on both staging and production servers.
 - Here is an example of a demo for one of main requirements:
 - Show the pipeline is triggered automatically by changes on Github.
 - Intentionally make an error of a change (like database connection error) and the website does not pass a test on the staging server and notify the team.
 - Make a change of the website without any error and show the website passes the test on staging server and continue to deploy the change on the production server.
 - Here is an example of a demo for one of advance requirements:
 - Demonstrate advanced deployment strategies in action to show your failure recovery mechanisms with some examples include:
 - Include a specific set of scenario tests to address system failures like server downtime, database connection issues, and failed deployments and automate responses to these scenario (e.g., redeploying containers or restarting failed services).
 - Demonstrate rollback after failed deployments and verify the application returns to a stable state. For example, if you implement either one of these strategies:
 - Blue-Green Deployment: Route traffic to a “green” environment after testing; revert to “blue” if issues arise.
 - Canary Deployment: Gradually release updates to a small subset, monitor performance, and halt/rollback on anomalies.
 - Try to simulate traffic spikes and showcase the scalability of the application (e.g., through horizontal or vertical scaling on the production server).
- The time allocation of two parts is up to you as long as the whole video must be within the time limit while the demo showcases most of the important required features of the system so you should prioritize the CI/CD pipeline demo.

- Each team member must present their own respective part of the project during the demo. This is a great opportunity for each team member to showcase the work they have done and demonstrate their understanding of the project. It is also an excellent opportunity for each team member to improve their presentation skills and gain experience working in a group. I would encourage each team member to review their work and prepare well before the demo to make the best impression.
-



Submission Requirements

- You should submit **one single zip file** to Canvas, named "**COSC2767_A2_[YourGroupName].zip**"
 - This is a group submission. One member (the Project Manager) submits on behalf of the team.
 - **Your submission zip file should contains:**
 1. **Your source code**
 - Organize your source code and resource files nicely and logically.
 - Your source code submission in the zip file should be the copy of the last commit on your Github repo.
 - This folder should contain all pipeline code (to setup, implement and maintain your pipeline): Jenkinsfile or equivalent, Dockerfiles, Ansible playbooks, CloudFormation templates, orchestration manifests, and your test suites. Include a README explaining how to run the pipeline from scratch.
 2. **Document.pdf** is the required report.
 3. **Github.txt** contains:
 - The full URL of your team's assigned private repository in the **rmit-vietnam-computing-technologies** organisation.
 4. **Youtube.txt** contains the link of the presentation video:
 - **Platform:** Upload your video to YouTube.
 - **Visibility:** The video **must** be set to **Unlisted**. This ensures it is not public but accessible to anyone with the link.
 - **Submission:** Do **not** upload the video file directly to Canvas. The platform is not optimized for video playback, which slows down the marking process.
 5. **COSC2767_Condition3_Declaration_[YourGroupName].pdf** (make sure it's a pdf file) which is required even if you did not use AI. The link is [here](#).
 - You can make multiple submissions on Canvas, but only the last one is graded thus it is better to submit early draft and continue to submit as you revise and improve the submission.
-

Deliverables

- Every file **your team wrote or substantially modified** must carry the comment header below. It is **not** required on the supplied RMIT Store application files you did not change - leaving those untouched is what lets a marker see which work is yours.
- Omit the header where the format does not support comments (JSON). In shell and Python scripts, the header goes immediately **after** the shebang line, not above it. In YAML, use "#".

```
RMIT University Vietnam
Course: COSC2767|COSC2805 Systems Deployment and Operations
Semester: 2026B
Assessment: Assignment 2
Author: Your name (e.g. Nguyen Van Minh)
ID: Your student id (e.g. 1234567)
Created date: dd/mm/yyyy (e.g. 31/07/2026)
Last modified: dd/mm/yyyy (e.g. 05/08/2026)
Acknowledgement: Acknowledge the resources that you use here.
```

- Test your submission carefully on one of the computers in the lab to resolve any unforeseen issues before submitting it.
-

Assigned GitHub Repository

- For this assignment, each team will be provided with an dedicated private repository in the following GitHub organization: **rmit-vietnam-computing-technologies**
- The teaching team will create the repository and add each member of your team as a collaborator. You do not need to create the repository yourself.
- Your repository name will follow this format: **<year>-<course>-<assignment>-<campus>-<your-group-name>**
- For example: **2026b-COSC2767-a2-sg-your-group-name**

Accepting Your Repository Invitation

After your repository has been created:

1. GitHub will send a collaborator invitation to the email address associated with your GitHub account.
2. Open the invitation email and accept the invitation.
3. You must accept the invitation within **seven days**. GitHub invitations expire after this period.
4. After accepting the invitation, confirm that you can access the private repository in the **rmit-vietnam-computing-technologies** organization. The link for the organization is <https://github.com/rmit-vietnam-computing-technologies/>. 
5. You should see your private repository in our organization.
6. Clone/Setup the assigned repository to your computer or EC2 and use it for all development work for this assignment.

Check your spam or junk-mail folder if you cannot find the invitation. If the invitation has expired, your GitHub username is incorrect, or you cannot access the repository, contact the teaching team as soon as possible.

Repository Requirements

- All project code must be committed and pushed to the private GitHub repository assigned to you by the teaching team.
- Do not create or use a personal public repository for this assignment.
- Do not publish, mirror or share your team's work in any repository other than the one assigned to you. Importing the supplied RMIT Store application into your assigned private repository is required and is not a breach of this rule - see "What your repository must contain" below.
- Work stored outside the assigned private repository may not be accepted for marking.
- The source code in your Canvas ZIP file must be identical to the source code in the final commit of the assigned repository.
- Include the complete URL of the assigned repository in **Github.txt**.

What your repository must contain

Must be present

- The supplied application, imported as above, with your test suites added.
- Every pipeline artefact: Jenkinsfile, Dockerfiles, compose files, Ansible playbooks and inventories, CloudFormation templates, orchestration manifests, monitoring and alerting configuration.
- **.env** files for every environment variable your pipeline needs, with placeholder values.
- A root **README.md** that lets a marker reproduce your pipeline on a clean AWS account: prerequisites, the order stacks and playbooks run in, required Jenkins credentials by ID, how to trigger a deploy, how to trigger a rollback.

The comment header on every file your team wrote or substantially modified. It is not required on the supplied application files you did not touch.

Must not be present

- **Build output:** for example, client/dist/, client/node_modules/, server/.venv/, server/staticfiles/, server/media/.
- The video file. YouTube link only.

Development-History Requirements

Your GitHub history is part of the assessment evidence. Markers will examine:

- the number of branches;
- the purpose of each branch;
- branch names;
- the number of commits;
- commit dates and timestamps;
- whether commits are reasonably distributed across the approximately the development period;
- whether the development appears incremental or was rushed into a short period immediately before submission;
- the clarity and meaning of commit messages;
- whether branches were merged logically;
- whether the repository shows an understandable progression from initial setup to the final application.

Commit and branch counts will be considered in relation to the scope and quality of the work. A large number of empty, trivial, duplicated, or artificially divided commits will not receive additional credit.

Use branches appropriately for separate features, fixes, refactoring work, or development stages.

Students are not expected to create a separate branch for every small change. Branches should have a clear purpose and should represent meaningful units of work.

Commit regularly throughout the project rather than uploading the complete project shortly before the deadline.

Teamwork

Team members must nominate a **Project Manager** and a **Technical Lead** for your team. The Project Manager is responsible for planning and controlling tasks in the project as well as chairing meetings and building a good team spirit. The Technical Lead is responsible for evaluating and making major technical decisions in the project. He/she is the gatekeeper of the GitHub repository of the project, determining what is accepted or rejected from the repository. In addition, the Technical Lead may coaches other members on specific technical tasks (design, coding, debugging, testing) when needed.

Peer Evaluation

Everyone in the team is expected to contribute a **fair share of work** in the project in order to receive the same marks as other members. If a member fails to do so, the Project Manager must inform the Lecturer **at least one week** before the assignment due for corrective actions.

The Lecturer reserves the right to adjust an individual team member mark if he/she fails to attend the required meetings and/or fails to contribute a fair share of work.

In fact, in the final submitted documentation, please include a table of team member contribution in term of percentage % to inform the lecturer about the contribution effort of each team members.

If all members work well together with equivalently contributed works, then the contribution should be equally divided (e.g. 25% for each member if this is group of 4, or 33.3% for group of 3, etc.). Otherwise, please discuss within your group to record the actual contributions of all members (e.g. a slightly higher percentage for people with more contribution and vice versa)

Note: the contribution should be considered overall in terms of initiative (help managing the project, contribute excellent ideas), amount of work, quality of work, and support for other members, etc.

Advice: Team members must nominate a Project Manager for your team. The Project Manager is responsible for planning and controlling tasks in the project as well as chairing meetings and building a good team spirit. The Technical Lead is responsible for evaluating and making major technical decisions in the project. He/she is the gatekeeper of the GitHub repository of the project, determining what is accepted or rejected from the repository. In addition, the Technical Lead may coaches other members on specific technical tasks (design, coding, debugging, testing) when needed.

AI Condition for this Assessment: Condition 3 - Bounded Process AI

This assessment uses **Condition 3: Bounded Process AI** from RMIT GenAI Assessment Framework.

1	NO AI IN ASSESSMENT	2	AI FOR SURFACE POLISH	3	AI FOR BOUNDED PROCESS	4	REQUIRED AI USE
PURPOSE Assure independent capability, judgement, reasoning, memory, live performance or disciplinary technique.	PURPOSE Allow light editing support without changing the substance of the student's thinking.	PURPOSE Allow the use of AI as a limited support tool while remaining the author and decision-maker.	PURPOSE AI use is part of the learning outcome. Students must demonstrate AI literacy, critique and professional judgement.				
STUDENT AI USE Not permitted for any part of the assessed work. Make clear this does not ban AI for general learning outside the task.	STUDENT AI USE Allowed only for grammar, spelling, clarity, formatting or language polish. No new ideas, code, analysis, sources or structure.	STUDENT AI USE Permitted only for specified processes, such as brainstorming, critique, feedback, comparison, explanation, debugging hints or alternative perspectives.	STUDENT AI USE Required in specified ways. The brief names the tool or tool type, required uses, limits & responsibilities.				
BEST FIT Core threshold skills; exams; live coding; oral defence; in-class design critique; tasks where personal competence must be evidenced.	BEST FIT Reports, reflections, design rationales, professional writing, where language polish should not be the assessed capability.	BEST FIT Design, research, ethics, coding, problem-solving, planning, critique, iteration and refinement tasks.	BEST FIT AI-assisted analysis, comparison of AI outputs, prompt iteration, critique of generated work, human-in-the-loop design, professional workflow tasks.				
DESIGN MOVES Supervised setting; unseen prompt/data/code; no internet or GenAI; short time window; personalised inputs; viva/demo where appropriate.	DESIGN MOVES Brief declaration; specify examples of allowed and prohibited use; ask students to keep drafts if needed.	DESIGN MOVES Clearly defined allowed uses, prohibited uses, and required evidence. Assess how students evaluate, adapt, reject or improve AI output.	DESIGN MOVES Assess both the main artefact and the AI process. Include an AI companion document as a graded component where appropriate.				
EVIDENCE Student's own reasoning trail: working, code, calculations, annotations, diagrams, test results, explanation, live responses.	EVIDENCE Short declaration: tool used, what it was used for, confirmation that content and decisions remain the student's.	EVIDENCE Declaration or AI-use doc: tool, purpose, prompt summary, output summary, what was accepted/rejected/revised, why.	EVIDENCE AI companion document: prompts/process, outputs used, critique, revisions, decisions, risks, acknowledgement and referencing.				
MARKER FOCUS Accuracy, reasoning, method, diagnosis, justification, independent decision-making.	MARKER FOCUS Assess disciplinary content. Do not reward polished wording if reasoning is weak.	MARKER FOCUS Quality of judgement, refinement beyond AI output, transparency, fit between AI use and task aims.	MARKER FOCUS Appropriateness of AI use, critique, iteration, ethical/professional judgement, quality of final human-authored work.				
WATCH FOR Do not use recall-only questions as the main assurance. Closed/no-AI still needs application and reasoning.	WATCH FOR Surface polish can easily drift into rewriting, restructuring or idea generation. Name that boundary clearly.	WATCH FOR Do not let the AI-use record become busywork. It should evidence thinking, not transcript dumping.	WATCH FOR Do not assess only the final product. Students must show how they used AI well not just the use of AI.				

Use of generative AI is optional. AI may be used only for the approved process-support purposes explained below. **It must not produce the submitted pipeline code or configuration, the infrastructure design, the report, the reflection, the presentation script, or any other final assessable work.**

For this assessment, "code" means **all submitted technical artefacts**, including but not limited to: Jenkinsfiles and pipeline scripts, Dockerfiles and docker-compose files, Ansible playbooks/roles/inventories, AWS CloudFormation (or equivalent IaC) templates, Kubernetes or Docker Swarm manifests, shell/Python/Node scripts, test suites (unit, integration, Web UI), monitoring and alerting configuration (e.g. Prometheus, Grafana, email notification config), and any modification made to the application codebase.

Every team must submit one completed **Condition 3 Bounded Process AI Declaration**, including teams in which no member used AI. The declaration is submitted once per team and must cover the AI use of all members.

You may use AI ONLY for the following approved process-based purposes:

- 1. Brainstorming at the start of the project** - generating candidate pipeline architectures, tool options, deployment strategies, or testing approaches to consider. The final architecture, tool selection, design decisions, and all implementation remain your team's own.
- 2. Learning DevOps concepts and tools.** You may use AI to explain concepts, documentation, error messages, syntax, or example configurations in simpler terms. This includes topics such as Jenkins

declarative vs scripted pipelines, Docker images and layers, Docker Swarm and Kubernetes, Ansible modules and idempotency, AWS EC2/VPC/IAM/CloudFormation, Git branching and webhook triggers, blue-green and canary deployments, Prometheus metrics and Grafana dashboards, and testing frameworks. You may **study** AI-generated examples or snippets for learning purposes, but you must not copy, paste, adapt, or otherwise incorporate AI-generated code or configuration into your project. All submitted code and configuration must be written and understood by you.

3. **Debugging hints** - asking AI to interpret an error message (for example a Jenkins console log, Docker build failure, Ansible task error, output, AWS stack event, or failing test output), explain possible causes, suggest diagnostic steps, or indicate where and why a fault may occur. AI may be used to help you understand the problem, but corrected or rewritten AI-generated code or configuration must not be copied into your project.
4. **Comparing alternative implementation approaches described in plain language** - for example Docker Swarm vs Kubernetes, blue-green vs canary deployment, Ansible vs shell provisioning, CloudFormation vs manual console setup, or Selenium vs Cypress for UI testing. You must make the decision, justify it in your own words, and write the implementation yourself.
5. **Receiving feedback on the clarity and structure of report text you wrote yourself.** AI may tell you what is unclear, disorganised, repetitive or missing. It must not supply replacement sentences, and you must not paste its wording back in. AI must not write, rewrite, expand, or paraphrase report content. Keep the draft you had **before** the feedback; you may be asked to produce it.
6. **Validating the syntax of a configuration file you wrote yourself** -for example asking whether the YAML indentation in your own playbook or manifest is well-formed, or whether your own JSON is valid. AI must not supply the corrected file, generate configuration content, or design the stages, tasks, resources, or parameters. The logic and content of every configuration file must be yours.
7. **Creating decorative or illustrative images for the report**, including through an AI image-generation tool. You remain responsible for selecting, editing, and appropriately using the final asset. Every AI-generated image must be labelled as such in **its own caption** as well as acknowledged in your report's Reference section - in a report full of genuine dashboards and consoles, a References-only acknowledgement is too easy to miss. This does **not** extend to your architecture or application flow diagram (see prohibitions below).
8. **Understanding the supplied RMIT Store codebase.** You may use AI to explain how the provided application works - what a settings module does, how the SPA reads "config.js" at page load, what a migration will change when applied, where an authorisation rule is enforced, what a management command does, or what a section of the README means. This includes opening the supplied application in an AI tool that can read the files, so long as the tool is in a **read-only or explanation mode. This purpose is "reading, not authoring"**: nothing the tool produces may be written into, or copied into, your assessment repository. **Understanding the application you are deploying is the point of the exercise, and you are encouraged to use AI for it.**

You must not use AI beyond these approved purposes. In particular, for this assessment you must NOT use AI to:

- **Generate any pipeline code, script, or configuration that is copied, pasted, or otherwise incorporated into your submission, in whole or in part.** This covers every artefact listed in the definition of "code" above.
 -
- AI coding assistants - including GitHub Copilot, Claude Code, Codex, Cursor, Windsurf, Gemini CLI, Amazon Q Developer, Tabnine and similar tools - must not write, edit, complete or refactor files in your assessment repository. **Inline completion and agent or auto-edit modes must be switched off in the editor you use for this assessment** (for example, disable GitHub Copilot in VS Code, or the AI Assistant plugin in JetBrains IDEs). Be prepared to demonstrate that setting if asked.
- These tools may be used **separately** for learning and for purpose 8 above. If you use an AI tool to read and explain the supplied application, do so on a checkout outside your working repository. The test is not which tool you opened; it is whether AI-produced code or configuration ended up in your submission. It must not.
 - **Write any part of your report**, including the problem statement, proposed solution description, tool and technology introductions, explanation of chosen settings and configurations, known bugs and problems, self-evaluation and reflection, reflection on the guest lecture, project responsibilities commentary, or conclusion.
 - **Generate your application flow or architecture diagram, or its content.** The diagram is assessable design work and must be produced by your team. Drawing tools (draw.io, Lucidchart, Excalidraw, PlantUML written by you, etc.) are permitted, with their AI diagram-generation features left unused. Several of these tools now ship a "generate diagram from a prompt" feature; using it on your architecture or pipeline diagram is a breach of this condition.
 - **Write, script, narrate, or generate your video presentation.** The presentation script and spoken commentary must be your own. AI-generated voiceovers, voice cloning, synthetic presenters or avatars must not be used. Team members must present in their own voice and appear as themselves.
 - **Fabricate or generate evidence.** All screenshots, logs, dashboards, test results, and demo footage must be genuine captures from your own running system. AI-generated or synthetically edited "proof" is a serious academic integrity breach.
 - **Misrepresent your repository history.** Commits must reflect the work actually done by the team member named in them.
 - **Generate images without acknowledging their use.** AI-generated images are permitted only as decorative or illustrative assets, and every AI-generated image must be acknowledged in your report's Reference section.
 - **Generate references or citations.** Fabricated or AI-invented sources will be treated as academic misconduct.

You remain responsible for all ideas, decisions, reasoning, content and evidence submitted for marking. **AI output must not replace your own thinking, judgement or authorship.**

Declaration requirement

Your team must submit **one** completed **Condition 3 Bounded Process AI Declaration** (linked below) with your assessment, even if no member used AI.

- **One declaration per team.** It is included once in the team's submission zip and is submitted by the Project Manager on behalf of the team.
- **It must be signed or acknowledged by all team members**, and it must list the names and student IDs of every member so that it is clear the whole team has agreed to its contents.
- If no member used AI, complete the section confirming that no AI was used.
- If any member used AI for approved process support, the declaration must record that use **member by member** — identifying who used which tool, what it was used for, what output was received, what was accepted, rejected or revised, and how the interaction influenced the final submission. A single vague team-level statement is not sufficient.
- Discuss and complete the declaration together as a team before submitting, in the same way you finalise the Project Responsibilities table.

If the team submits its assessment work without the required declaration, the whole assessment will be treated as incomplete. The submission date will be the date on which both the assessment work and the completed declaration have been submitted.

Plagiarism

Plagiarism is a form of cheating. It is the presentation of the work, idea or creation of another person as though it is your own.

This is a group assessment. Collaboration between members of your officially registered group is expected and required.

However, your group must not share assessment work, source code, configuration, reports, diagrams, scripts, test suites, infrastructure definitions or other assessable material with students outside your group, and you must not use another group's work as your own.

Where external resources are used, they must be appropriately acknowledged as required by RMIT's academic integrity requirements and by the instructions in this assessment.

Any submission containing material presented as the group's own work when it was produced by another person, another group, or an unauthorised source may be treated as an academic integrity breach.

Please ask your Lecturer if you are uncertain whether a form of collaboration or reuse is permitted.

The penalty for plagiarism is extremely severe at RMIT. If you are found to plagiarize, you will receive no marks for an assessment or even an entire course. Repeated plagiarism will lead to exclusion from RMIT.

For more information, visit RMIT's website for [Academic integrity information and policy](#) .

Support Resources

This assessment requires that you meet RMIT's expectations for academic integrity. More information and advice on how to avoid plagiarism are available in the Getting Started module.

Open the [Academic Integrity](#)  page

Additional library and learning resources are available to help with the assessment in this course

Link to [Assignment Support](#).

Criteria	Ratings					Pts
Main Requirements All required features are implemented correctly in all use cases for the pipeline: - CI/CD pipeline correctly integrated with AWS and containerized microservices. - Automatic CI process triggered by GitHub updates. - Successful continuous delivery with testing server first, then automated deployment to production upon passing tests. - Robust testing framework with unit, integration, and UI tests that cover critical functionalities and show reliability. - Effective use of Jenkins (or similar) for	10 to >8.0 pts Excellent All criteria fully met and functionally correct with almost no improvements identified.	8 to >6.0 pts Very Good Almost all criteria fully met and functionally correct with 1 improvements identified.	6 to >4.0 pts Good Some criteria fully met and functionally correct with 2 improvements identified.	4 to >2.0 pts Fair Some criteria fully met and functionally correct with 3 improvements identified.	2 to >0 pts Need improvement Few criteria met and functionally correct with 4 or more improvements identified.	10 pts

Criteria	Ratings					Pts
automation and Ansible (or similar) for configuration management. - Automated alerts set up for any build, test, or deployment failure. - Proper orchestration with Docker Swarm or Kubernetes ensuring scalability and reliability.						
Advance Requirements All required features are implemented correctly in all use cases for the pipeline: - Infrastructure as Code is used comprehensively to provision and manage all required infrastructure. - Monitoring and alerting integrated for real-time insights, with	7 to >5.6 pts Excellent All criteria fully met and functionally correct with almost no improvements identified.	5.6 to >4.2 pts Very Good Almost all criteria fully met and functionally correct with 1 improvements identified.	4.2 to >2.8 pts Good Some criteria fully met and functionally correct with 2 improvements identified.	2.8 to >1.4 pts Fair Some criteria fully met and functionally correct with 3 improvements identified.	1.4 to >0 pts Need improvement Few criteria met and functionally correct with 4 or more improvements identified.	7 pts

Criteria	Ratings					Pts
clear dashboards and alerts. - Advanced deployment strategies demonstrated effectively and thoroughly tested. - Additional enhancements beyond the given scope that significantly improve automation, efficiency, reliability, or scalability.						
Report (Written Document) - Clear, comprehensive, and professional report. - Well-structured document with all required sections and easy-to-follow formatting. - Clear explanations of the pipeline's architecture, tools, and features.	6 to >4.8 pts Excellent All criteria fully met and functionally correct with almost no improvements identified.	4.8 to >3.6 pts Very Good Almost all criteria fully met and functionally correct with 1 improvements identified.	3.6 to >2.4 pts Good Some criteria fully met and functionally correct with 2 improvements identified.	2.4 to >1.2 pts Fair Some criteria fully met and functionally correct with 3 improvements identified.	1.2 to >0 pts Need improvement Few criteria met and functionally correct with 4 or more improvements identified.	6 pts

Criteria	Ratings					Pts
- Screenshots and diagrams included for clarity and as proof of meeting requirements. - Logical reasoning behind chosen configurations and tools. - Strong reflection on CI/CD industry practices and lessons learned. - No major grammatical, formatting, or structural issues.						
Video Demonstration & Presentation - Engaging, clear, and concise video demonstration: - Within the time limit and well-organized flow (presentation followed by demo). - Each main and advanced feature clearly demonstrated, including	7 to >5.6 pts Excellent All criteria fully met and functionally correct with almost no improvements identified.	5.6 to >4.2 pts Very Good Almost all criteria fully met and functionally correct with 1 improvements identified.	4.2 to >2.8 pts Good Some criteria fully met and functionally correct with 2 improvements identified.	2.8 to >1.4 pts Fair Some criteria fully met and functionally correct with 3 improvements identified.	1.4 to >0 pts Need improvement Few criteria met and functionally correct with 4 or more improvements identified.	7 pts

Criteria	Ratings					Pts
triggering the pipeline, showing test results, and verifying successful deployments. - Effective use cases and scenarios simulated (e.g., intentional errors, rollbacks, scalability tests). - All team members contribute and demonstrate a good understanding of their part.						
Code Quality, GitHub Repository and Submission Organisation - Follow to best practices and coding standards with consistent formatting. - Meaningful commit messages and descriptive comments in code.	5 to >4.0 pts Excellent All criteria fully met and functionally correct with almost no improvements identified.	4 to >3.0 pts Very Good Almost all criteria fully met and functionally correct with 1 improvements identified.	3 to >2.0 pts Good Some criteria fully met and functionally correct with 2 improvements identified.	2 to >1.0 pts Fair Some criteria fully met and functionally correct with 3 improvements identified.	1 to >0 pts Need improvement Few criteria met and functionally correct with 4 or more improvements identified.	5 pts

Criteria	Ratings					Pts
- Code is readable, modular, and easily understandable. - Well-organized submission files and structure. - Correct folder structure, all required files included. - Proper documentation headers in source code files. - Submission matches the requested format (zip file with README, links, and report included). - Appropriate use of GitHub repository (no personal repos) as per instructions. - GitHub development practice: Branches and commits are meaningful, clearly named, appropriately used, and show						

Criteria	Ratings					Pts
regular development across the four-week project period rather than a rushed final upload. - The README is clear, contains team details, repo URL, and unlisted YouTube link.						
Total Points: 35						